

Desenvolvimento e Validação de um Ambiente de Simulação para Teleoperação de um Rover Inspirado no Curiosity Utilizando CoppeliaSim e Python

Gustavo Spoti Costa, *Discente*, gustavo-s-c@aluno.edu.com.br, Engenharia Mecatrônica, UFSJ; **Filipe Augusto Santos Rocha**, *Docente Orientador*, filipe@ufsj.edu.br, Departamento DETEM, UFSJ

Resumo—A crescente utilização de sistemas robóticos em ambientes extremos impulsiona o desenvolvimento de arquiteturas capazes de realizar operações remotas seguras, estáveis e eficientes. Nesse contexto, foi desenvolvido e validado um ambiente de simulação para teleoperação de um rover inspirado no Curiosity da NASA utilizando o software CoppeliaSim integrado a uma API desenvolvida em Python. O sistema permite o controle remoto da locomoção e de um manipulador robótico por meio de um joystick, possibilitando análise do comportamento dinâmico do rover em diferentes cenários simulados. A arquitetura implementada utiliza comunicação contínua entre o simulador e a interface de controle, incluindo técnicas de suavização de sinais, limitação de aceleração e controle adaptativo de direção, visando aumentar a estabilidade e o realismo da operação. Foram realizados testes envolvendo terrenos planos e irregulares, análise do desempenho temporal do sistema e comparação entre diferentes engines físicas disponíveis no ambiente de simulação. Os resultados demonstraram funcionamento estável da arquitetura proposta, comportamento coerente da suspensão rocker-bogie e operação consistente do manipulador robótico durante os experimentos. A comparação entre engines físicas evidenciou que a escolha da engine afeta principalmente a locomoção do rover, sendo Newton e ODE as engines com melhor desempenho, sem impacto perceptível sobre a operação do manipulador robótico. Apesar das limitações relacionadas à ausência de dados físicos precisos do rover original e da execução local da comunicação o sistema mostrou-se adequado como plataforma experimental para estudos em teleoperação, robótica móvel e gêmeos digitais aplicados à exploração espacial, devido à estabilidade apresentada durante os testes, à consistência da arquitetura de controle e à capacidade de reproduzir comportamentos dinâmicos coerentes em diferentes condições de operação.

Palavras-chave—simulação; controle remoto; exploração planetária; interface homem-máquina; suspensão rocker-bogie.

I. INTRODUÇÃO

A exploração de ambientes extremos impulsiona o desenvolvimento de sistemas robóticos capazes de operar remotamente, reduzindo riscos humanos e ampliando a execução de tarefas em locais hostis. Superfícies planetárias, áreas industriais de risco elevado e regiões subterrâneas exigem soluções com elevada mobilidade, capacidade de sensoriamento e manipulação robótica [1].

No setor aeroespacial, missões como o programa Artemis evidenciam a importância de rovers na exploração e no suporte operacional em ambientes de difícil acesso. Esses sistemas precisam executar deslocamentos em terrenos irregulares, coleta de amostras e manipulação de objetos sob restrições severas de comunicação e operação [2].

A operação remota desses robôs depende de arquiteturas capazes de fornecer controle preciso, previsível e confiável. Entretanto, fatores como latência, perda de comunicação e limitações sensoriais dificultam a interação entre operador e sistema robótico [3]. Nesse cenário, ambientes de simulação, incluindo abordagens baseadas em gêmeos digitais, tornam-se ferramentas relevantes para validação de estratégias de controle, treinamento operacional e análise de desempenho antes da aplicação em sistemas físicos [4].

Plataformas como o CoppeliaSim permitem modelagem robótica, simulação física e integração com aplicações externas, favorecendo o desenvolvimento de arquiteturas experimentais de controle remoto. A utilização dessas ferramentas reduz custos de desenvolvimento e aumenta a segurança durante etapas iniciais de validação [5].

Diante do contexto apresentado, este trabalho tem como objetivo geral desenvolver e validar um ambiente de simulação integrado a uma API em Python para controle remoto de um rover inspirado no Curiosity utilizando joystick [10]. Como objetivos específicos, busca-

se avaliar a viabilidade da arquitetura desenvolvida, analisar o comportamento dinâmico do sistema e investigar a resposta do manipulador robótico e da suspensão rocker-bogie em diferentes condições simuladas.

II. REVISÃO BIBLIOGRÁFICA

A. Teleoperação e robótica remota

A teleoperação consiste no controle de sistemas robóticos à distância por meio de um operador humano, permitindo a execução de tarefas em ambientes inacessíveis ou perigosos. Essa abordagem pode ser entendida como uma extensão da capacidade humana, na qual o operador interage com o ambiente remoto por meio de interfaces responsáveis pela transmissão de comandos e recebimento de informações de telemetria [9]. A teleoperação é amplamente utilizada em aplicações industriais, exploração espacial, operações submarinas e ambientes extremos, nos quais a presença humana direta é inviável ou apresenta riscos elevados.

No contexto da robótica moderna, a interação entre humanos e robôs evoluiu para modelos mais colaborativos, nos quais o operador não apenas controla o sistema, mas também supervisiona e ajusta seu comportamento. Entretanto, a teleoperação ainda apresenta desafios relacionados à comunicação, percepção e tomada de decisão, especialmente em cenários de elevada complexidade operacional [3].

Entre os principais desafios da teleoperação destacam-se a latência de comunicação, a perda de qualidade do feedback sensorial e as limitações na precisão dos comandos [12]. Esses fatores podem comprometer o desempenho do sistema e influenciar a interação entre operador e robô, evidenciando a importância do desenvolvimento e da validação de arquiteturas de controle remoto em ambientes simulados.

B. Ambientes de simulação robótica

O uso de ambientes de simulação tornou-se uma etapa fundamental no desenvolvimento e validação de sistemas robóticos. A simulação permite testar algoritmos, estratégias de controle e arquiteturas de comunicação em ambientes controlados, reduzindo custos operacionais e riscos associados à implementação em sistemas reais. Além disso, desempenha papel importante na avaliação de desempenho e identificação de falhas antes da aplicação prática [6].

A simulação também oferece vantagens como repetibilidade de experimentos, controle de variáveis e facilidade de modificação de cenários, tornando-se uma ferramenta relevante em pesquisas e desenvolvimento tecnológico [6]. Nesse cenário, plataformas como o CoppeliaSim

destacam-se pela flexibilidade e capacidade de integração com diferentes linguagens de programação e sistemas externos [5].

O CoppeliaSim, anteriormente denominado V-REP, permite a modelagem de sistemas robóticos complexos, incluindo sensores, atuadores e manipuladores, além de oferecer suporte à integração com APIs externas e execução contínua de simulações [5]. Essas características favorecem a criação de ambientes de teste voltados para aplicações em robótica móvel e teleoperação.

C. Integração com APIs e controle externo

A integração entre ambientes de simulação e sistemas de controle externo representa um aspecto fundamental no desenvolvimento de aplicações robóticas modernas. O uso de APIs (*Application Programming Interfaces*) permite a comunicação entre diferentes componentes do sistema, viabilizando arquiteturas distribuídas e modulares.

Nesse contexto, a linguagem Python destaca-se pela simplicidade, versatilidade e ampla disponibilidade de bibliotecas voltadas ao desenvolvimento de aplicações robóticas. Sistemas como o ROS (*Robot Operating System*) utilizam arquiteturas baseadas em comunicação distribuída, permitindo a troca de informações entre diferentes módulos por meio de protocolos padronizados [7].

A utilização de arquiteturas cliente-servidor possibilita o controle remoto de simuladores, nos quais o ambiente de simulação atua como servidor e os módulos externos enviam comandos e recebem dados continuamente. Essa abordagem favorece a escalabilidade do sistema e permite integração com diferentes dispositivos de entrada, como joysticks e interfaces personalizadas.

D. Interfaces homem-máquina (HMI)

As interfaces homem-máquina desempenham papel fundamental na teleoperação de sistemas robóticos, sendo responsáveis pela comunicação entre o operador e o sistema controlado. Dispositivos como joysticks e gamepads são amplamente utilizados devido à ergonomia, facilidade de uso e capacidade de fornecer controle intuitivo sobre movimentos e ações do robô.

A eficiência da interação humano-robô depende da qualidade da interface utilizada, incluindo fatores como tempo de resposta, precisão dos comandos e clareza do feedback fornecido ao operador. Interfaces bem projetadas reduzem a carga cognitiva e aumentam a eficiência operacional em ambientes complexos [8].

E. Aplicações em exploração espacial

A aplicação de sistemas de teleoperação em exploração espacial representa um dos principais campos de desenvolvimento da robótica moderna. Missões conduzidas pela NASA utilizam rovers para exploração de superfícies planetárias, coleta de amostras e execução de experimentos científicos em ambientes inóspitos [11].

O rover Curiosity, pertencente à missão Mars Science Laboratory, exemplifica a utilização de sistemas robóticos avançados operados remotamente, combinando mobilidade, manipulação e sensores para execução de tarefas complexas em Marte. Nessas operações, a comunicação entre a Terra e o robô apresenta atrasos significativos, exigindo estratégias de controle parcialmente autônomas e planejamento antecipado de ações [9], [12].

A operação de rovers em ambientes espaciais envolve desafios relacionados à latência, limitações de comunicação e necessidade de robustez operacional, reforçando a importância do uso de ambientes simulados para validação e testes prévios de sistemas robóticos [5].

III. METODOLOGIA

A. Visão geral da metodologia

O presente trabalho consiste no desenvolvimento de um modelo virtual do rover Curiosity no ambiente de simulação CoppeliaSim, com controle em execução contínua por meio de um joystick, utilizando uma interface desenvolvida em Python. O sistema permite o controle da velocidade e da direção das rodas, bem como a movimentação do braço robótico do rover, possibilitando a avaliação do comportamento dinâmico do modelo sob diferentes condições de operação.

A abordagem metodológica adotada caracteriza-se como uma simulação computacional com coleta e análise de dados. O foco do trabalho está na construção do cenário virtual, integração entre o hardware de entrada, representado pelo controle de Xbox One, e o software de controle, bem como na obtenção de métricas relacionadas ao desempenho do sistema, como tempo de resposta, posição das juntas e sinais de entrada provenientes do controle.

A escolha dessa abordagem foi motivada pela disponibilidade de ferramentas acessíveis, pela flexibilidade oferecida por ambientes de simulação e pela relevância do tema para o desenvolvimento de competências em robótica e sistemas embarcados. Adicionalmente, a utilização do CoppeliaSim foi sugerida como uma alternativa viável para exploração de conceitos relevantes na área, sem a necessidade de implementação física do sistema.

Para o desenvolvimento do projeto, foram utilizadas as ferramentas CoppeliaSim, Visual Studio Code, Python,

Pygame e Matplotlib. O ambiente de desenvolvimento Visual Studio Code foi utilizado para a implementação e depuração do código-fonte em Python. O CoppeliaSim foi empregado como ambiente de simulação física, responsável pela modelagem do rover e execução das interações dinâmicas. A linguagem Python foi utilizada para a criação de uma interface de programação (API) responsável pela comunicação entre o controle e o simulador. A biblioteca Pygame foi empregada para a leitura das entradas do controle Xbox One, enquanto o Matplotlib foi utilizado para a geração de gráficos e análise dos dados coletados.

B. Modelagem e configuração física do rover no CoppeliaSim

O modelo do rover foi construído com base em informações disponibilizadas pela NASA [11], buscando representar sua estrutura mecânica e sistema de locomoção do tipo rocker-bogie. O modelo tridimensional utilizado na simulação foi obtido a partir do repositório oficial da NASA¹. Foram configuradas as juntas e propriedades físicas no ambiente de simulação, permitindo a reprodução aproximada do comportamento do sistema real.

A Figura 1 apresenta a estrutura hierárquica desenvolvida para o modelo do rover no ambiente CoppeliaSim. O modelo tridimensional obtido já possuía seus principais componentes estruturais organizados em partes independentes, incluindo os elos do manipulador robótico, os elementos da suspensão rocker-bogie, as rodas e os suportes responsáveis pelo esterçamento. Essa divisão facilitou a definição da hierarquia dos objetos e a configuração das juntas, permitindo representar as relações cinemáticas e dinâmicas entre o chassi, a suspensão, o sistema de direção, as rodas e o manipulador robótico.

A partir do chassi principal, definido como elemento raiz da hierarquia, foram conectados os conjuntos da suspensão rocker-bogie de ambos os lados do veículo, além do manipulador robótico. Essa organização facilita a propagação dos movimentos entre os componentes e permite a configuração independente das propriedades físicas e dos modos de controle associados a cada subsistema.

C. Sistema de suspensão rocker-bogie

A mobilidade do rover foi baseada em uma suspensão do tipo rocker-bogie, amplamente utilizada em veículos

¹Disponível em: <https://science.nasa.gov/resource/curiosity-rover-3d-model/>. Acesso em: jul. 2026.

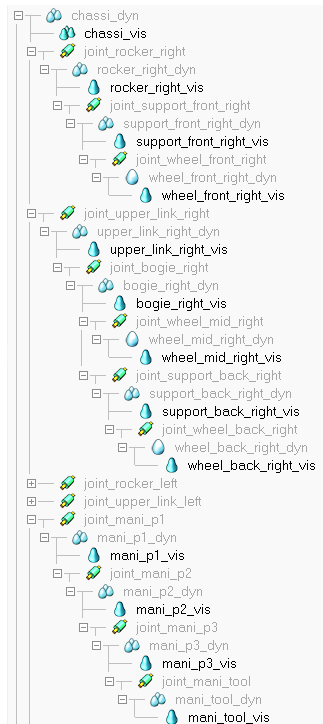


Figura 1. Estrutura hierárquica do modelo do rover no CoppeliaSim

destinados à exploração planetária devido à sua capacidade de transpor obstáculos e manter contato contínuo das rodas com o solo.

A Figura 2 apresenta os principais elementos da suspensão implementada no modelo virtual.

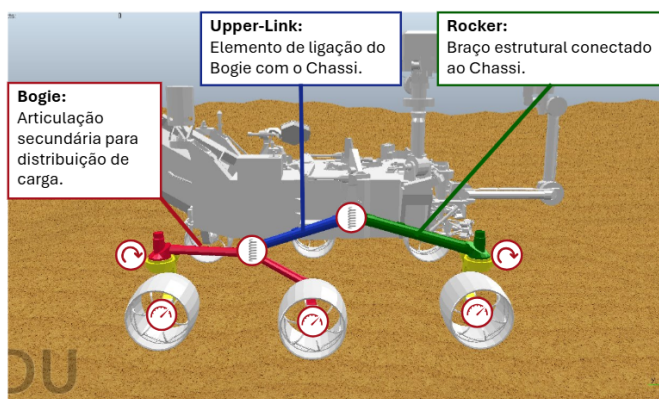


Figura 2. Identificação dos principais componentes da suspensão rocker-bogie utilizada no modelo virtual. O rocker está destacado em verde, o upper link em azul, o bogie em vermelho, os suportes de direção em amarelo e as rodas em branco.

O sistema é composto pelos elementos rocker, bogie e upper link, além dos suportes de direção e das rodas. O rocker, destacado em verde na Figura 2, consiste no principal braço estrutural da suspensão, responsável por conectar o conjunto ao chassi do rover e transmitir as cargas provenientes do terreno.

O bogie, destacado em vermelho, conecta as rodas central e traseira de cada lado do veículo, permitindo que ambas se adaptem de forma independente às irregularidades do terreno. Essa configuração contribui para a distribuição das cargas e para a manutenção da estabilidade durante a locomoção.

O upper link, representado em azul, atua como um elemento auxiliar de ligação entre o bogie e a região central do sistema de suspensão. Sua função é complementar a estrutura mecânica da suspensão e contribuir para a distribuição dos esforços gerados durante a navegação sobre superfícies irregulares.

Os suportes de direção, destacados em amarelo, foram posicionados junto às rodas dianteiras e traseiras. Esses componentes são responsáveis pelo esterçamento das rodas, permitindo a realização de curvas e manobras durante a operação do rover.

As rodas, representadas em branco, constituem os elementos de contato direto com o terreno. Cada roda foi conectada à suspensão por meio de juntas dinâmicas com controle de velocidade, possibilitando a aplicação dos comandos de tração gerados pela arquitetura de controle desenvolvida em Python.

Para reproduzir o comportamento elástico observado durante a interação com terrenos irregulares, as articulações entre rocker, bogie, upper link e chassi foram configuradas utilizando juntas dinâmicas operando em modo mola (*spring*). Essa configuração permitiu que a suspensão respondesse às irregularidades do relevo, absorvendo parte das deformações impostas pelo terreno e retornando gradualmente à posição de equilíbrio após a remoção das cargas externas.

A utilização dessa arquitetura teve como objetivo reproduzir, de forma simplificada, características importantes do sistema rocker-bogie empregado em rovers de exploração planetária, incluindo adaptação passiva ao terreno, distribuição de carga entre as rodas e contribuição para a estabilidade durante a navegação em superfícies irregulares, conforme avaliado na Seção de Resultados e Discussão.

A Tabela I apresenta as principais juntas utilizadas no modelo e seus respectivos tipos.

Embora todas as juntas do modelo sejam de revolução, diferentes estratégias de controle foram empregadas de acordo com a função desempenhada por cada componente. Essa abordagem permitiu reproduzir comportamentos distintos para locomoção, direção, suspensão e manipulação robótica.

A Tabela II apresenta os modos de operação utilizados para cada grupo de juntas.

Por fim, as juntas do manipulador robótico foram configuradas em modo dinâmico com controle de posição.

Tabela I

PRINCIPAIS JUNTAS UTILIZADAS NO MODELO DO ROVER

Junta	Tipo	Grupo
joint_rocker_right/left	Revolução	Suspensão
joint_upper_link_right/left	Revolução	Suspensão
joint_bogie_right/left	Revolução	Suspensão
joint_support_front_right/left	Revolução	Direção
joint_support_back_right/left	Revolução	Direção
joint_wheel_front_right/left	Revolução	Tração
joint_wheel_mid_right/left	Revolução	Tração
joint_wheel_back_right/left	Revolução	Tração
joint_mani_p1	Revolução	Manipulador
joint_mani_p2	Revolução	Manipulador
joint_mani_p3	Revolução	Manipulador
joint_mani_tool	Revolução	Manipulador

Tabela II

CONFIGURAÇÃO DINÂMICA DAS JUNTAS DO MODELO

Grupo	Tipo de Controle
Tração	Controle de velocidade
Direção	Controle de posição
Suspensão	Mola
Manipulador	Controle de posição

Essa estratégia permitiu controlar individualmente cada grau de liberdade do braço, possibilitando movimentos suaves e previsíveis durante a operação remota do sistema.

A combinação dessas diferentes configurações permitiu representar, em ambiente virtual, os principais comportamentos mecânicos esperados para o rover, servindo como base para os experimentos de teleoperação e para a avaliação do desempenho dinâmico apresentados na Seção de Resultados e Discussão.

D. Arquitetura de controle e integração entre sistemas

A integração entre os sistemas foi realizada por meio de uma API desenvolvida em Python, que realiza a leitura contínua das entradas do controle e as converte em comandos de velocidade e posição. Esses comandos são enviados ao Coppeliasim, que executa a simulação física do rover com base nos parâmetros definidos.

O sistema opera em execução contínua, permitindo interação direta do usuário com o modelo simulado. A organização hierárquica adotada permitiu separar adequadamente os subsistemas de locomoção, suspensão e manipulação, facilitando tanto a configuração dos parâmetros físicos quanto a implementação da arquitetura de controle. Além disso, essa estrutura busca contribuir para a estabilidade da simulação e simplificar futuras modificações e expansões do modelo virtual.

O controle do sistema segue um fluxo no qual os sinais do joystick são capturados, processados pela API e

convertidos em comandos aplicados às juntas do modelo no simulador. Esse processo permite a movimentação do rover e de seu braço robótico de forma interativa, conforme ilustrado na Figura 3.



Figura 3. Diagrama do fluxo de sinais do sistema

A API foi organizada em blocos funcionais que separam claramente as etapas de configuração, inicialização, execução contínua e pós-processamento do sistema. Um conjunto de funções auxiliares, incluindo a curva logística de suavização, a função de limitação de valores (clamp) e a função de atualização de velocidade com limitação de aceleração, foi implementado de forma independente dos modos de operação, sendo reaproveitado tanto no controle da locomoção quanto no controle do manipulador robótico. Os parâmetros de calibração do sistema, como os limites de *deadzone*, os ganhos da curva logística, as velocidades e acelerações máximas e os limites angulares de cada junta, foram centralizados em uma seção específica do código, permitindo o ajuste do comportamento do rover sem a necessidade de alterar a lógica de controle propriamente dita.

Na etapa de inicialização, a API estabelece a conexão remota com o Coppeliasim por meio da função `simxStart` e obtém, junto ao simulador, os identificadores (handles) de cada junta do modelo, correspondentes às rodas, ao sistema de direção e ao manipulador robótico, os quais são posteriormente utilizados para o envio dos comandos de controle. Em seguida, o sistema entra em um laço de repetição principal, executado continuamente até o encerramento da simulação, no qual os eixos e botões do controle são lidos a cada iteração por meio da biblioteca Pygame. A cada ciclo, o botão Start é monitorado para alternância entre os modos de locomoção e manipulação, enquanto os sinais dos eixos correspondentes ao modo ativo são processados conforme descrito na seção de Processamento dos sinais e modos de operação e convertidos em comandos de velocidade ou posição, enviados ao Coppeliasim em modo de comunicação contínua (streaming).

Durante a execução do laço principal, a API também é responsável pelo registro simultâneo dos dados de operação, incluindo o tempo de simulação, os sinais de entrada do joystick, os comandos aplicados às juntas, a taxa de atualização do laço (FPS) e o tempo de resposta

entre o comando enviado e o movimento efetivo da roda. Ao término da simulação, esses dados são processados para o cálculo das métricas estatísticas apresentadas na seção de Resultados e Discussão e utilizados na geração automática dos gráficos por meio da biblioteca Matplotlib, permitindo a análise posterior do comportamento dinâmico do sistema.

A Figura 4 ilustra a arquitetura da API desenvolvida, destacando seus principais blocos funcionais e o fluxo de informações entre a aquisição dos comandos do joystick, o processamento dos sinais, a comunicação com o CoppeliaSim e o registro dos dados da simulação.

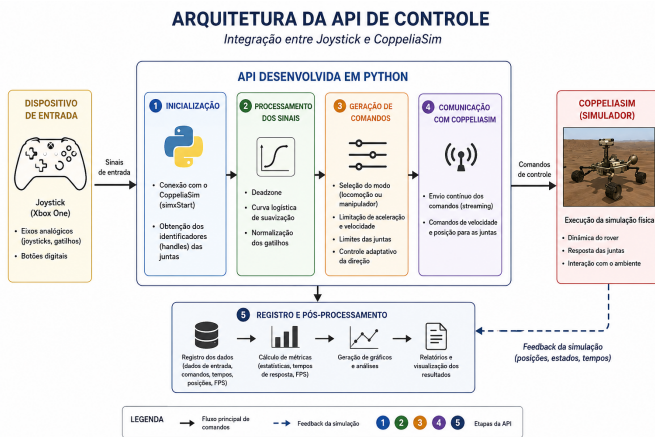


Figura 4. Arquitetura da API desenvolvida em Python para integração entre o joystick e o ambiente de simulação CoppeliaSim.

E. Processamento dos sinais e modos de operação

O código desenvolvido realiza o processamento dos sinais de entrada por meio de três mecanismos complementares. Primeiro, aplica-se uma zona morta (*deadzone*) de 0,1 sobre os eixos analógicos do joystick, eliminando pequenas variações registradas quando o controle encontra-se em posição neutra. Em seguida, os valores dos eixos são processados por uma função de suavização logística, definida como

$$f(x) = \frac{2}{1 + e^{-kx}} - 1, \quad (1)$$

com $k = 0,5$, a qual reduz a sensibilidade do controle próximo à origem e proporciona uma transição mais gradual entre os valores de entrada e os comandos aplicados ao sistema. Por fim, os sinais provenientes dos gatilhos analógicos, fornecidos pela biblioteca Pygame no intervalo $[-1, 1]$, são normalizados para o intervalo $[0, 1]$, adequando-os ao uso como fatores multiplicativos nos comandos de acionamento da ferramenta do manipulador. Esse processamento visa garantir maior estabilidade e controle sobre os movimentos do rover.

Foram implementados dois modos de operação distintos: um voltado à locomoção do robô e outro dedicado à manipulação do braço robótico. No modo de locomoção, os comandos provenientes do joystick controlam a velocidade das rodas e o sistema de direção do veículo. No modo manipulador, os comandos são direcionados às juntas responsáveis pela movimentação do braço robótico.

A Figura 5 apresenta o mapeamento dos comandos do controle Xbox One referentes ao modo de locomoção. O eixo vertical do analógico esquerdo é responsável pela geração do comando de velocidade linear das rodas, enquanto o eixo horizontal do analógico direito determina o comando de esterçamento. A Figura 6, por sua vez, apresenta o mapeamento correspondente ao modo de manipulação: o eixo horizontal do analógico esquerdo controla a rotação da base (*joint_mani_p1*), o eixo vertical do mesmo analógico controla a elevação do braço (*joint_mani_p2*), o eixo vertical do analógico direito controla o movimento do cotovelo (*joint_mani_p3*), e os gatilhos RT/LT² são responsáveis pela rotação do efetador (*joint_mani_tool*) em sentidos opostos. Em ambas as figuras, o botão central superior indica a alternância entre os dois modos de operação.

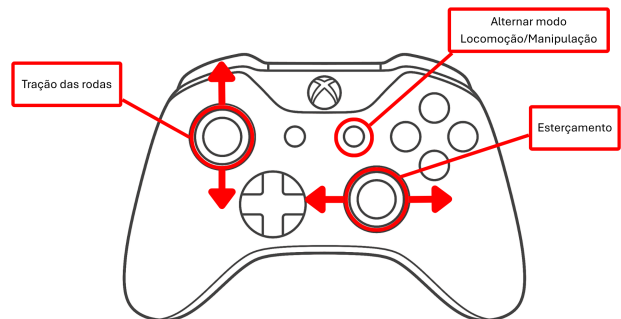


Figura 5. Mapeamento dos comandos do controle Xbox para o modo de locomoção do rover.

A Tabela III apresenta as funções atribuídas a cada junta do manipulador robótico.

Com o objetivo de garantir estabilidade durante a teleoperação e evitar movimentos incompatíveis com a estrutura mecânica do modelo, foram definidos limites angulares e velocidades máximas para as juntas do manipulador robótico. As juntas *joint_mani_p1* e *joint_mani_p2* foram limitadas aos intervalos de $\pm 1,00$ rad e $\pm 1,05$ rad, respectivamente. A junta *joint_mani_p3* foi configurada para operar entre $-0,52$ rad e $1,92$

²Os gatilhos analógicos RT e LT estão localizados na parte superior traseira do controle, atrás dos botões indicados na Figura 6, não sendo visíveis na vista frontal ilustrada.

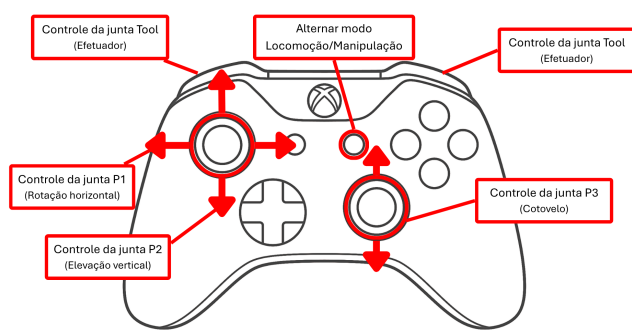


Figura 6. Mapeamento dos comandos do controle Xbox para o modo de manipulação do braço robótico.

rad, enquanto a junta *joint_mani_tool* foi limitada ao intervalo entre 0 rad e 1,92 rad. Para todas as juntas do manipulador foi adotada velocidade angular máxima de 1,5 rad/s. Adicionalmente, o sistema de direção foi limitado a um ângulo máximo de esterçamento de $\pm 0,5$ rad, contribuindo para a estabilidade da navegação durante manobras. Esses parâmetros foram implementados na arquitetura de controle desenvolvida em Python e utilizados durante toda a execução da simulação.

Os limites operacionais adotados para cada junta são apresentados na Tabela IV.

Tabela III
FUNÇÕES DAS JUNTAS DO MANIPULADOR ROBÓTICO

Junta	Função
<i>joint_mani_p1</i>	Rotação horizontal da base
<i>joint_mani_p2</i>	Elevação do braço
<i>joint_mani_p3</i>	Movimento do cotovelo
<i>joint_mani_tool</i>	Rotação da ferramenta

Tabela IV
LIMITES OPERACIONAIS DAS JUNTAS CONTROLADAS

Junta	Limite (rad)	Vel. Máx. (rad/s)
<i>joint_mani_p1</i>	-1,00 a 1,00	1,5
<i>joint_mani_p2</i>	-1,05 a 1,05	1,5
<i>joint_mani_p3</i>	-0,52 a 1,92	1,5
<i>joint_mani_tool</i>	0,00 a 1,92	1,5
Direção	-0,50 a 0,50	1,5

Durante a execução da simulação, o sistema registra os dados de entrada e saída, que posteriormente são utilizados para análise e visualização por meio de gráficos.

F. Procedimento de validação

Para validação do sistema, foram realizados testes em cenário virtual contendo terrenos irregulares, com o objetivo de observar o comportamento da suspensão

rocker-bogie, a estabilidade do modelo e a distribuição do centro de massa durante a locomoção, considerando os parâmetros físicos estimados adotados na modelagem.

Além da avaliação qualitativa do comportamento mecânico do rover, foram gerados gráficos relacionando os sinais de entrada do joystick com as respostas do sistema. As métricas analisadas incluíram tempo de resposta, posição das juntas do manipulador e comandos enviados ao simulador.

A análise dos resultados foi realizada por meio dos dados registrados pela API e da utilização da biblioteca Matplotlib para geração dos gráficos correspondentes.

A validação do modelo foi realizada de forma interna, sem comparação direta com dados experimentais reais, devido à limitação de informações disponíveis sobre o rover original.

G. Limitações da metodologia

Entre as principais limitações do trabalho destaca-se a ausência de dados precisos sobre propriedades físicas do rover, como distribuição de massa, coeficientes de atrito e características dos materiais empregados em sua construção, fatores que impactam a fidelidade da simulação.

Adicionalmente, não foi implementado um sistema de controle avançado, como controladores do tipo PID, limitando a precisão e a robustez do controle em determinadas situações operacionais.

Dessa forma, o modelo desenvolvido representa uma aproximação funcional do rover Curiosity, permitindo estudos de teleoperação e comportamento dinâmico em ambiente virtual, porém não podendo ser caracterizado como um gêmeo digital completo devido às simplificações adotadas.

IV. RESULTADOS E DISCUSSÃO

A. Validação funcional do sistema de controle remoto

O sistema de controle remoto desenvolvido foi integrado com sucesso ao ambiente de simulação CoppeliaSim, permitindo o controle remoto do rover por meio de um joystick. Durante os testes realizados, o sistema apresentou funcionamento contínuo e estável, sem ocorrência de travamentos ou falhas de comunicação entre a API desenvolvida e o ambiente de simulação.

O rover foi capaz de executar movimentos de locomoção para frente e para trás, bem como realizar mudanças de direção por meio do esterçamento das rodas dianteiras e traseiras. O sistema também possibilitou a alternância entre o modo de controle da locomoção e o modo de operação do braço robótico, permitindo ao

operador controlar diferentes funcionalidades do rover utilizando o mesmo dispositivo de entrada.

Além da movimentação da plataforma móvel, o manipulador robótico operou corretamente durante os experimentos, permitindo a execução de movimentos independentes em suas juntas. O sistema foi capaz de alternar entre os modos de controle sem interrupções perceptíveis ao operador, mantendo estabilidade operacional durante toda a execução da simulação.

Durante os experimentos, verificou-se que a utilização de técnicas de suavização dos sinais de entrada contribuiu para aumentar a estabilidade e previsibilidade da movimentação do rover. Conforme apresentado no Algoritmo 1, o sinal do joystick passa por duas etapas de tratamento independentes entre si. Primeiramente, é aplicada uma zona morta (*deadzone*), que atribui valor nulo ao eixo sempre que sua magnitude estiver abaixo do limite definido, eliminando pequenas variações indesejadas do sinal próximas à posição neutra do joystick. Em seguida, independentemente do resultado da etapa anterior, é aplicada uma curva logística de suavização, condicionada a um parâmetro de habilitação, que reduz a sensibilidade excessiva próxima à região central dos controles, permitindo maior precisão em movimentos de baixa velocidade.

Algoritmo 1 Aplicação da curva logística aos sinais do joystick

Require: Valor do eixo do joystick x ; ganho de suavização $k = 0,5$; limiar de zona morta *deadzone* = 0,1

- 1: Ler o valor do eixo do joystick x
 - 2: **if** $|x| < \text{deadzone}$ **then**
 - 3: $x \leftarrow 0$
 - 4: **end if**
 - 5: **if** suavização habilitada **then**
 - 6: $x \leftarrow \frac{2}{1 + e^{-kx}} - 1$
 - 7: **end if**
 - 8: Utilizar x como entrada do controlador
-

Adicionalmente, foram implementados mecanismos artificiais de limitação de aceleração e desaceleração, conforme apresentado no Algoritmo 2, buscando simular comportamento dinâmico mais próximo de um sistema físico real. Essa abordagem evitou mudanças bruscas de velocidade e contribuiu para uma movimentação mais suave do rover durante a simulação.

Observou-se que, quando o sistema de limitação de aceleração era removido, o rover passava a responder instantaneamente aos comandos enviados pelo joystick, produzindo acelerações bruscas e dificultando o controle preciso da plataforma. Nessas condições, durante mudanças rápidas de direção o sistema apresentava

Algoritmo 2 Limitação artificial da aceleração

Require: Velocidade atual v ; velocidade desejada v_d ; aceleração máxima a_{max} ; desaceleração máxima d_{max} ; intervalo de tempo dt

- 1: Calcular velocidade desejada v_d
 - 2: $\Delta v \leftarrow v_d - v$
 - 3: **if** $|v_d| > |v|$ **then**
 - 4: $L \leftarrow a_{max} \cdot dt$
 - 5: **else**
 - 6: $L \leftarrow d_{max} \cdot dt$
 - 7: **end if**
 - 8: Limitar Δv ao intervalo $[-L, L]$
 - 9: $v \leftarrow v + \Delta v$
 - 10: Enviar velocidade aos motores
-

comportamento mais agressivo e menos previsível. Em contrapartida, a implementação do controle de aceleração progressiva permitiu reproduzir um comportamento mais próximo de veículos reais, aumentando a estabilidade operacional e a sensação de inércia durante a condução.

B. Análise do desempenho temporal

A estabilidade temporal do sistema foi avaliada por meio da medição da taxa de atualização da simulação e do tempo de resposta computacional entre os comandos enviados pelo joystick e o processamento realizado pela API integrada ao simulador. A taxa de atualização foi obtida diretamente do ambiente Coppeliasim, por meio de duas fontes independentes de medição, com o objetivo de caracterizar de forma confiável o desempenho do motor de simulação física, isolando-o do overhead computacional introduzido pela comunicação cliente-servidor e pelo processamento da API externa.

A primeira medição, denominada *FPS de cena*, foi obtida por meio da função *simxGetLastCmdTime*, que retorna o tempo de simulação associado ao último comando processado pelo servidor do Coppeliasim. A variação desse tempo entre iterações consecutivas do laço principal de controle foi utilizada para o cálculo do FPS, refletindo a taxa de avanço do tempo simulado percebida pela API externa. A segunda medição, denominada *FPS via script Lua*, foi obtida por meio de um Child Script anexado à cena, responsável por medir o tempo real (*wall-clock*) decorrido entre passos de atuação consecutivos utilizando a função interna *sim.getTime*. Esse valor foi publicado como um sinal e lido pela API Python por meio da função *simxGetFloatSignal*, representando uma medição obtida diretamente no interior do motor de simulação. A utilização de duas fontes independentes permitiu uma validação cruzada do desempenho temporal do sistema.

- FPS de cena: 20,00
- FPS médio via script Lua: 20,92
- FPS mínimo via script Lua: 8,40
- FPS máximo via script Lua: 1000,00³
- Tempo de cena total: 53,750 s
- Tempo real total: 60,367 s
- Fator de tempo real (cena/real): 0,890
- Tempo médio de atualização: 0,005915 s
- Tempo mínimo de atualização: 0,005001 s
- Tempo máximo de atualização: 0,025153 s
- Desvio padrão temporal: 0,001002 s

Os resultados demonstraram que a taxa de atualização de cena manteve-se perfeitamente constante ao longo de toda a execução, refletindo o passo de simulação fixo do motor de física. O fator de tempo real obtido, de 0,890, indica que a simulação transcorreu a aproximadamente 89% da velocidade do tempo real, apresentando um pequeno atraso acumulado em relação ao relógio de parede, compatível com o overhead computacional inerente à execução simultânea da física, da interface gráfica e da comunicação com a API externa. Esse resultado é relevante para aplicações de teleoperação, uma vez que valores próximos da unidade indicam que a dinâmica percebida pelo operador corresponde de forma fiel à dinâmica temporal real do sistema simulado. Já o baixo desvio padrão observado no tempo de atualização refere-se especificamente ao tempo de resposta entre o comando do joystick e o movimento efetivo da roda, indicando comportamento consistente da latência de atuação do sistema, o que favorece aplicações de controle remoto em execução contínua.

A Figura 7 apresenta a comparação direta entre as duas metodologias de medição sobre o mesmo eixo temporal. Observa-se que o FPS de cena manteve-se estável em torno de 20 FPS durante toda a execução, ao passo que o FPS medido pelo script Lua, com média de 20,92 FPS, apresentou maior variabilidade, oscilando entre um mínimo de 8,40 FPS e picos pontuais de curtíssima duração. Esses picos, associados a intervalos de tempo extremamente pequenos entre chamadas consecutivas do script, são atribuídos a instantes específicos de inicialização do sinal e não representam o comportamento típico do sistema. De maneira geral, a proximidade entre a média do FPS via script Lua e o valor nominal de 20 FPS confirma a consistência entre as duas metodologias de medição, reforçando a validade da taxa de atualização obtida por meio da função *simxGetLastCmdTime* como

³Valor associado a um pico pontual de curtíssima duração, provavelmente decorrente de um intervalo de tempo próximo de zero entre chamadas consecutivas do script logo após sua inicialização, não sendo representativo do comportamento típico do sistema.

representativa do desempenho real do motor de simulação.

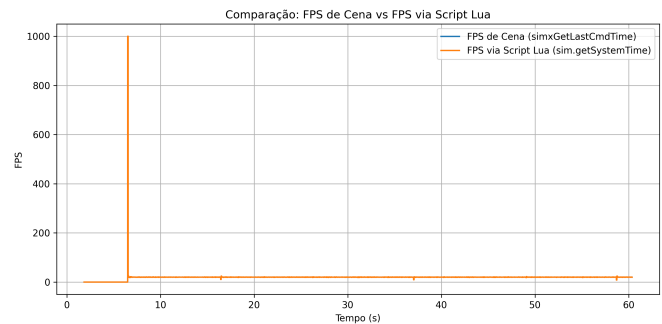


Figura 7. Comparação entre o FPS de cena medido via *simxGetLastCmdTime* e o FPS medido internamente pelo script Lua anexado à cena

Apesar da baixa latência computacional observada, o comportamento dinâmico percebido pelo operador apresentou respostas mais suaves devido à implementação de técnicas artificiais de aceleração progressiva e amortecimento de movimento, utilizadas para aumentar o realismo da simulação.

A Figura 8 apresenta a relação entre o sinal de entrada do joystick e a velocidade aplicada ao rover durante os testes realizados.

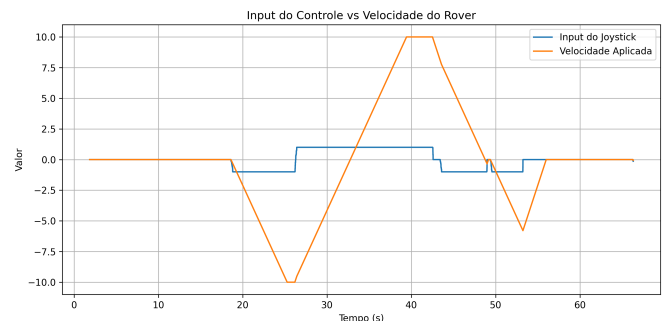


Figura 8. Comparação entre o sinal de entrada do joystick e a velocidade aplicada ao rover

Observa-se que a velocidade do rover não varia instantaneamente em relação ao sinal de entrada, apresentando comportamento progressivo de aceleração e desaceleração. Esse comportamento foi intencionalmente implementado para simular efeitos de inércia e atrito, aumentando o realismo da movimentação da plataforma móvel.

C. Controle de direção e estabilidade

O sistema de direção do rover foi implementado utilizando esterçamento das rodas dianteiras e traseiras, permitindo maior capacidade de manobra durante a locomoção. Adicionalmente, foi desenvolvido um mecanismo

adaptativo de controle direcional, no qual a velocidade angular de esterçamento foi reduzida automaticamente em altas velocidades.

Essa abordagem teve como objetivo aumentar a estabilidade dinâmica do rover durante movimentos rápidos, reduzindo mudanças bruscas de direção e contribuindo para um comportamento mais previsível do sistema. A lógica empregada para esse ajuste adaptativo é apresentada no Algoritmo 3.

Algoritmo 3 Controle adaptativo do esterçamento

Require: Comando de direção u_d ; velocidade linear atual v ; velocidade linear máxima v_{max} ; velocidade angular base de esterçamento $v_{steer,base}$; aceleração angular base $a_{steer,base}$

- 1: Ler comando de direção u_d
- 2: Calcular fator de velocidade

$$f \leftarrow \max \left(0, 2, 1 - \frac{|v|}{v_{max}} \right)$$

- 3: $v_{steer,max} \leftarrow v_{steer,base} \cdot f$
- 4: $a_{steer,max} \leftarrow a_{steer,base} \cdot f$
- 5: **if** $|u_d| > 0,05$ **then**
- 6: $\omega_d \leftarrow -u_d \cdot v_{steer,max}$
- 7: $\Delta\omega \leftarrow \omega_d - \omega$
- 8: Limitar $\Delta\omega$ ao intervalo $[-a_{steer,max} \cdot dt, a_{steer,max} \cdot dt]$
- 9: $\omega \leftarrow \omega + \Delta\omega$
- 10: **end if**

Além do controle adaptativo, foi implementado um mecanismo automático de retorno do esterçamento para a posição central utilizando um controlador proporcional e derivativo simplificado. O funcionamento desse controlador é apresentado no Algoritmo 4.

Algoritmo 4 Retorno automático do esterçamento

Require: Ângulo de direção atual θ ; velocidade angular ω ; ganho proporcional $K_p = 25,0$; ganho derivativo $K_d = 8,0$; ângulo máximo de esterçamento $\theta_{max} = 0,5$ rad

- 1: **if** não houver comando de direção **then**
- 2: $a \leftarrow -K_p\theta - K_d\omega$
- 3: $\omega \leftarrow \omega + a \cdot dt$
- 4: **end if**
- 5: $\theta \leftarrow \theta + \omega \cdot dt$
- 6: Limitar θ ao intervalo $[-\theta_{max}, \theta_{max}]$
- 7: Aplicar θ às juntas de direção

A combinação entre o ajuste adaptativo da sensibilidade da direção e o retorno automático do esterçamento contribuiu para reduzir oscilações e mudanças bruscas

de trajetória durante os testes realizados. Em velocidades elevadas, a limitação da velocidade angular de esterçamento tornou o comportamento do rover mais previsível, enquanto o controlador proporcional-derivativo promoveu o retorno suave da direção à posição central quando o operador interrompia o comando de esterçamento.

A Figura 9 apresenta a comparação entre o input de direção fornecido pelo controle e o ângulo de esterçamento resultante aplicado às rodas durante os experimentos realizados.

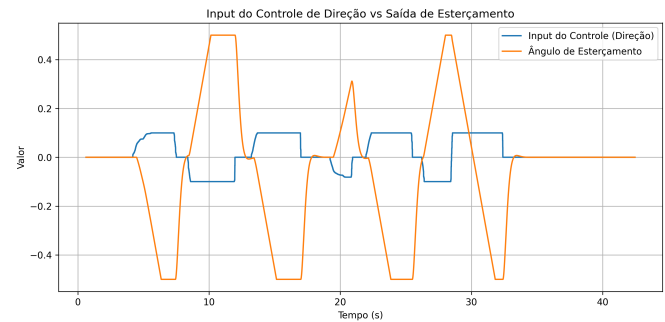


Figura 9. Input do controle de direção comparado ao ângulo de esterçamento das rodas ao longo do tempo

Observa-se que o sistema apresentou limitação angular estável e retorno suave para a posição central após a remoção do comando de direção. O pequeno sobresinal observado durante o retorno indica comportamento semelhante a sistemas amortecidos reais, contribuindo para maior realismo da simulação.

D. Controle do manipulador robótico

O manipulador robótico implementado possui quatro graus de liberdade, permitindo movimentos de rotação horizontal, elevação vertical, articulação intermediária e orientação do efetuador.

A figura 10 apresenta a identificação das juntas do manipulador robótico.

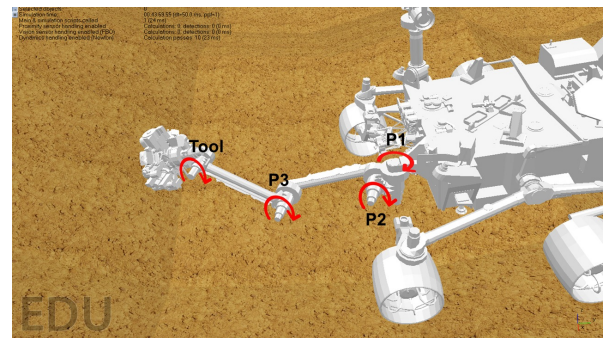


Figura 10. Manipulador robótico em operação no cenário virtual

A junta P1 foi responsável pela rotação horizontal da base do braço robótico, permitindo movimentação

lateral do manipulador. A junta P2 realizou a elevação vertical do braço, enquanto a junta P3 atuou como uma articulação intermediária semelhante a um cotovelo robótico. Por fim, a junta Tool foi utilizada para controlar a orientação vertical do efetuador.

Durante os testes realizados, o braço robótico apresentou comportamento estável e previsível, sendo capaz de executar movimentos contínuos dentro dos limites definidos para cada junta. Foram implementadas restrições angulares para impedir movimentos fora dos limites físicos estabelecidos no modelo virtual.

Observou-se que, ao atingir seus limites angulares máximos, as juntas interrompiam o movimento com estabilidade, sem apresentar oscilações, vibrações ou recuos indesejados. Esse comportamento contribuiu para aumentar a previsibilidade do sistema e evitar instabilidades durante a operação do manipulador.

O sistema de controle do manipulador utilizou suavização dos sinais analógicos do joystick e limitação de velocidade angular, reduzindo movimentos bruscos e aumentando a precisão operacional durante a movimentação das juntas.

O controle das juntas do manipulador foi implementado utilizando integração numérica simples baseada em velocidade angular. Essa abordagem foi adotada devido ao foco principal do trabalho estar voltado à validação da arquitetura de controle remoto e da integração entre o joystick, a API em Python e o ambiente de simulação.

A utilização de controladores PID exigiria modelagem mais precisa da dinâmica das juntas, incluindo parâmetros físicos como inércia, atrito e torque dos atuadores, os quais não estavam disponíveis com elevado grau de fidelidade no modelo adotado. Adicionalmente, como o sistema operava em ambiente totalmente simulado e com baixa latência computacional, a utilização de técnicas simplificadas de suavização e limitação de aceleração mostrou-se suficiente para garantir estabilidade operacional e movimentação previsível das juntas durante os testes realizados.

A Figura 11 apresenta a evolução temporal das posições angulares das juntas do manipulador robótico.

Os resultados demonstraram estabilidade das posições angulares ao longo do tempo, sem ocorrência de oscilações significativas ou ruídos perceptíveis nos sinais das juntas. Adicionalmente, verificou-se que cada junta operou de forma independente, respeitando os limites físicos definidos durante a modelagem do sistema.

E. Comparação entre engines físicas

Durante o desenvolvimento do projeto, diferentes engines físicas disponíveis no CoppeliaSim foram avaliadas

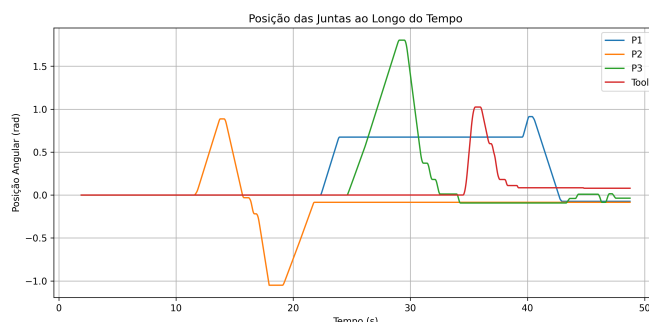


Figura 11. Posição angular das juntas do manipulador ao longo do tempo

com o objetivo de analisar diferenças no comportamento dinâmico do rover.

As engines testadas foram Bullet 2.78, Bullet 2.83, ODE e Newton, avaliadas em três cenários: deslocamento em terreno plano, deslocamento em terreno irregular e operação do manipulador robótico, este último contemplando o acionamento das juntas de forma isolada e conjunta.⁴

A engine Bullet 2.78 apresentou instabilidades já em terreno plano, com o rover deslizando mesmo sem o envio de comandos, além de leve deformação da suspensão em sentido não condizente com a movimentação das juntas e um travamento breve e quase imperceptível das rodas. Ainda assim, o rover respondeu corretamente a todos os comandos de deslocamento e esterçamento. Em terreno irregular, a suspensão e o esterçamento se adaptaram adequadamente ao relevo, porém os problemas identificados em terreno plano se mantiveram, com o deslizamento tornando-se ainda mais evidente. No teste do manipulador, o comportamento o esperado, sem travamentos, deformações ou violação dos limites configurados.

A versão Bullet 2.83 apresentou resultados ainda mais instáveis em terreno plano, com deformação da suspensão superior à observada na versão 2.78 e travamento mais acentuado das rodas, chegando a impedir momentaneamente a movimentação do rover; mesmo assim, os comandos de deslocamento longitudinal e de esterçamento foram atendidos. Em terreno irregular, os mesmos problemas se mantiveram e foram intensificados pelo relevo, a ponto de o rover ficar preso em alguns momentos, embora a suspensão e o esterçamento tenham se adaptado corretamente às irregularidades do terreno. No teste do manipulador, o comportamento ocorreu como o

⁴Vídeo comparativo do desempenho do rover nas engines físicas avaliadas, contemplando os três cenários de teste, disponível em: <https://youtu.be/LFJZ8wvNDwA?si=kjcsSZdWWTaLPe5N>.

esperada, sem travamentos, deformações ou violação dos limites configurados.

A engine ODE apresentou comportamento estável em terreno plano, atendendo aos comandos de deslocamento longitudinal e esterçamento, sem qualquer travamento das rodas ou deformação da suspensão. Em terreno irregular, manteve a ausência de deformações e travamentos, com apenas um episódio pontual de deslizamento do rover; a suspensão se adaptou bem às irregularidades e o esterçamento operou normalmente. No teste do manipulador, o comportamento também foi o esperado, sem travamentos, deformações ou violação dos limites configurados.

A engine Newton apresentou, em terreno plano, resultados equivalentes aos da ODE, com resposta estável aos comandos de deslocamento e esterçamento e ausência de travamentos ou deformações. Em terreno irregular, também não apresentou deformações ou travamentos, porém demonstrou maior incidência de deslizamento do rover em comparação à ODE, ainda que a suspensão tenha se adaptado adequadamente às irregularidades do terreno e o esterçamento tenha funcionado. No teste do manipulador, o comportamento foi equivalente ao das demais engines, sem travamentos, deformações ou violação dos limites configurados.

De maneira geral, o teste do manipulador não evidenciou diferenças perceptíveis entre as engines avaliadas, indicando que a escolha da engine física impactou principalmente o comportamento de locomoção do rover, sobretudo em relação a deslizamento, deformação da suspensão e travamento das rodas.

Os resultados obtidos demonstraram que a escolha da engine física influencia principalmente o comportamento de locomoção do sistema robótico simulado, afetando estabilidade, resposta da suspensão e qualidade da interação entre rodas e terreno, sem impacto perceptível sobre a operação do manipulador robótico. Entre as engines avaliadas, ODE e Newton apresentaram os resultados mais estáveis, com uma pequena vantagem da ODE em terreno irregular devido à menor incidência de deslizamento observada.

F. Limitações observadas

Apesar dos resultados satisfatórios obtidos, algumas limitações foram identificadas durante o desenvolvimento do sistema. A principal limitação está relacionada à ausência de dados físicos precisos do rover original, o que exigiu aproximações para parâmetros como distribuição de peso e propriedades dos materiais.

O controle implementado baseou-se predominantemente em técnicas simplificadas de suavização de sinais, limitação de aceleração e amortecimento artificial, complementadas por um controlador proporcional-derivativo (PD) simplificado empregado especificamente no retorno automático do esterçamento à posição central, conforme descrito no Algoritmo 4. Essa combinação mostrou-se suficiente para validação funcional do controle remoto em cenário virtual.

Embora controladores PID completos sejam amplamente utilizados em aplicações robóticas reais devido à sua capacidade de reduzir erro de regime, oscilações e sobresinais, sua implementação adequada exige modelagem dinâmica mais precisa do sistema físico, incluindo parâmetros como torque, atrito, resposta dos motores e características dos atuadores. Como o projeto possui foco principal na validação da integração computacional e da arquitetura de controle remoto, optou-se por uma abordagem simplificada de controle na maior parte dos subsistemas, com exceção do controlador PD simplificado já mencionado.

Assim, a implementação de controladores PID foi considerada uma possibilidade de expansão futura do projeto para aplicações envolvendo maior fidelidade dinâmica, controle físico embarcado e aproximação mais realista ao conceito de gêmeo digital.

Outra limitação observada refere-se à ausência de interação física avançada do manipulador com objetos do ambiente, uma vez que o modelo utilizado não possui mecanismo funcional de prensão.

Também foram observadas limitações associadas ao comportamento das engines físicas, especialmente em relação à estabilidade da suspensão e à interação das

Tabela V

RESUMO COMPARATIVO DO COMPORTAMENTO DAS ENGINES FÍSICAS AVALIADAS

Engine	Terreno plano	Terreno irregular	Manipulador
Bullet 2.78	Deslizamento sem comando; leve deformação da suspensão; travamento breve das rodas	Mesmos problemas do terreno plano, com deslizamento mais evidente	Comportamento normal
Bullet 2.83	Deformação e travamento mais acentuados; rover chegou a parar momentaneamente	Problemas intensificados; rover ficou preso em alguns momentos	Comportamento normal
ODE	Estável, sem travamento ou deformação	Estável, com um episódio pontual de deslizamento	Comportamento normal
Newton	Estável, sem travamento ou deformação	Estável, porém com deslizamento mais frequente que a ODE	Comportamento normal

rodas com o terreno. Essas diferenças demonstram que a fidelidade da simulação depende da engine utilizada e dos parâmetros físicos configurados no ambiente virtual.

G. Discussão dos resultados obtidos

Os resultados obtidos demonstraram que o sistema desenvolvido foi capaz de validar a proposta de controle remoto de um rover móvel em cenário virtual, permitindo integração funcional entre o joystick, a API desenvolvida em Python e o ambiente de simulação CoppeliaSim. A arquitetura implementada apresentou operação consistente durante os testes, permitindo controle contínuo da locomoção e do manipulador robótico sem ocorrência de falhas críticas ou interrupções na comunicação entre os módulos do sistema.

Apesar dos resultados satisfatórios, verificou-se que o comportamento temporal do sistema não representa integralmente as limitações encontradas em aplicações reais de teleoperação. Como toda a comunicação foi executada localmente na mesma máquina, a latência observada durante os testes apresentou valor médio de 6 ms, não sendo identificados atrasos perceptíveis durante a operação do sistema. Em aplicações reais de exploração remota, especialmente em cenários espaciais, atrasos de comunicação representam um dos principais desafios operacionais, afetando a precisão do controle e a tomada de decisão do operador.

Nesse cenário, o ambiente CoppeliaSim mostrou-se uma ferramenta adequada para o desenvolvimento do projeto, oferecendo recursos robustos para modelagem, configuração física e integração externa. A plataforma disponibiliza múltiplas engines físicas, ferramentas automáticas para cálculo de propriedades dinâmicas e suporte à importação de modelos tridimensionais externos, permitindo elevado grau de flexibilidade durante a construção do ambiente virtual. Adicionalmente, a possibilidade de integração com APIs externas facilitou a implementação da arquitetura de controle remoto proposta.

A utilização da linguagem Python também se mostrou adequada para o desenvolvimento da API de controle devido à ampla disponibilidade de bibliotecas voltadas para robótica, processamento de sinais e integração com dispositivos externos. A simplicidade da linguagem contribuiu para acelerar o desenvolvimento do sistema e facilitar a implementação de funções relacionadas ao controle do rover e do manipulador robótico. Entretanto, embora Python apresente elevada flexibilidade e facilidade de desenvolvimento, aplicações reais embarcadas podem exigir linguagens com maior desempenho computacional e menor *overhead* de execução, como C ou C++.

A interface homem-máquina baseada em joystick apresentou desempenho satisfatório durante os experimentos, oferecendo quantidade adequada de entradas analógicas e digitais para controle das funcionalidades do rover. A utilização dos analógicos para controle de velocidade e direção, juntamente com os gatilhos e botões auxiliares para operação do manipulador, permitiu a construção de uma interface relativamente intuitiva para o operador. No entanto, verificou-se que um sistema dedicado especificamente para exploração robótica poderia oferecer ergonomia superior, maior precisão operacional e melhor distribuição das funções de controle.

O comportamento dinâmico do rover apresentou características compatíveis com sistemas de suspensão do tipo rocker-bogie durante a navegação em terrenos irregulares. O modelo demonstrou capacidade de adaptação passiva ao relevo, mantendo tração satisfatória durante a travessia de obstáculos e inclinações moderadas. Entretanto, algumas limitações físicas ainda foram observadas, especialmente relacionadas à distribuição de massa do modelo virtual. Durante determinadas situações com o braço robótico totalmente estendido, o chassi apresentava inclinação excessiva para frente ou para trás, indicando necessidade de ajustes adicionais nas propriedades físicas e distribuição de peso do modelo.

Os testes realizados também demonstraram que a escolha da engine física influencia a resposta do sistema simulado, principalmente na locomoção do rover. As engines ODE e Newton apresentaram os resultados mais estáveis e realistas entre as avaliadas, sem deformações de suspensão ou travamento das rodas, sendo que a ODE apresentou uma ligeira vantagem em terreno irregular por evidenciar menor incidência de deslizamento em relação à Newton. Já as versões Bullet 2.78 e 2.83 apresentaram instabilidades relevantes, incluindo deformações na suspensão e travamento das rodas durante a navegação, sendo esses problemas mais severos na versão 2.83, a ponto de comprometer momentaneamente a movimentação do rover. Vale destacar que essas diferenças entre engines se restringiram à locomoção, uma vez que o comportamento do manipulador robótico se manteve consistente independentemente da engine utilizada.

Embora o sistema desenvolvido apresente potencial para futura implementação física, o estágio atual do projeto ainda caracteriza uma plataforma experimental de validação em ambiente virtual. A evolução para um rover físico exigiria refinamento significativo do modelo computacional, incluindo melhor aproximação das propriedades físicas do sistema real, implementação de sensores embarcados e desenvolvimento de arquiteturas de controle mais robustas.

Para que o modelo desenvolvido possa se aproximar

mais do conceito de gêmeo digital, seriam necessários dados mais precisos relacionados à estrutura interna do rover, distribuição de massa, propriedades dos materiais e comportamento dinâmico dos componentes mecânicos. A ampliação do número de graus de liberdade do manipulador robótico, a implementação de interação física avançada com objetos e a inclusão de comunicação remota real também contribuiriam para aumentar a fidelidade do sistema desenvolvido. Nesse mesmo sentido, futuras melhorias podem incluir a implementação de controladores avançados, como PID, integração com algoritmos de navegação autônoma e utilização de técnicas baseadas em inteligência artificial para assistência operacional e tomada de decisão.

V. CONCLUSÃO

O trabalho atingiu o objetivo proposto ao desenvolver e validar uma arquitetura de controle remoto para um rover inspirado no Curiosity em ambiente de simulação. A integração entre o modelo robótico, o joystick e a API desenvolvida permitiu executar, de forma consistente, comandos de locomoção e manipulação durante os testes realizados.

Os resultados demonstraram comportamento dinâmico coerente do sistema em diferentes cenários, incluindo deslocamento sobre terrenos irregulares e operação do manipulador robótico. A análise comparativa entre engines físicas evidenciou diferenças relevantes na resposta da suspensão rocker-bogie e na interação do rover com o terreno, sendo ODE e Newton as engines com melhor desempenho na locomoção, sem impacto observado sobre a operação do manipulador robótico. Técnicas de suavização de entrada, deadzone e aceleração progressiva contribuíram para reduzir oscilações bruscas e melhorar a previsibilidade do controle.

Como principal contribuição, o projeto estabelece uma plataforma experimental voltada ao controle remoto de rovers móveis utilizando ferramentas acessíveis e amplamente difundidas na área de robótica. O trabalho também consolida uma base para estudos relacionados a gêmeos digitais, validação de arquiteturas de controle remoto e simulação de sistemas robóticos aplicados à exploração espacial.

Entre as limitações identificadas, destacam-se a ausência de dados físicos detalhados do rover original, simplificações presentes no modelo tridimensional utilizado e a execução local da comunicação, que reduz os efeitos de latência observados em aplicações reais. Esses fatores limitam a fidelidade dinâmica da simulação e a representação completa das condições operacionais de um sistema físico.

Como continuidade do projeto, recomendam-se estudos envolvendo controle PID, inclusão de modelos físicos mais precisos, simulação de atrasos de comunicação e implementação de algoritmos de navegação autônoma. Também se mostra relevante a futura integração da arquitetura desenvolvida com plataformas robóticas físicas, permitindo validar o sistema em condições reais de operação.

REFERÊNCIAS

- [1] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*, 2nd ed., Springer, 2016.
- [2] National Aeronautics and Space Administration, “Artemis Program,” 2023. Available: <https://www.nasa.gov/artemis> Accessed: May 2026.
- [3] T. Fong, C. Thorpe, and C. Baur, “Robot, Asker of Questions,” *Robotics and Autonomous Systems*, vol. 42, no. 3–4, pp. 235–243, 2003.
- [4] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital Twin in Industry: State-of-the-Art,” *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [5] E. Rohmer, S. P. N. Singh, and M. Freese, “V-REP: A Versatile and Scalable Robot Simulation Framework,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013, pp. 1321–1326.
- [6] N. Koenig and A. Howard, “Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.
- [7] M. Quigley et al., “ROS: An Open-Source Robot Operating System,” in *ICRA Workshop on Open Source Software*, 2009.
- [8] J. Drury, J. Scholtz, and H. Yanco, “Awareness in Human-Robot Interactions,” in *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, 2003, pp. 912–918.
- [9] T. B. Sheridan, *Telerobotics, Automation, and Human Supervisory Control*, MIT Press, 1992.
- [10] National Aeronautics and Space Administration (NASA), “Curiosity Rover 3D Model”. Disponível em: <https://science.nasa.gov/resource/curiosity-rover-3d-model/>. Acesso em: 22 jun. 2026.
- [11] National Aeronautics and Space Administration (NASA), “Mars Science Laboratory (MSL) Curiosity”. Disponível em: <https://science.nasa.gov/mission/msl-curiosity/>. Acesso em: 23 jun. 2026.
- [12] S. B. Kamtam, Q. Lu, F. Bouali, O. C. L. Haas e S. Birrell, “Network Latency in Teleoperation of Connected and Autonomous Vehicles: A Review of Trends, Challenges, and Mitigation Strategies,” *Sensors*, v. 24, n. 12, art. 3957, 2024. DOI: 10.3390/s24123957.